

PAC-Opt v2.1

Prime-Aperiodic, Certified (Sentinel) Optimizer Wrapper

A closed-loop stability architecture for stochastic optimization under periodic forcing and other training-time cyclicities.

One-paragraph abstract (paper-ready)

Training pipelines often contain hidden periodicities (data ordering, evaluation cadence, augmentation cycles, distributed synchronization) that can induce **resonance** and increase **tail risk** (rare but severe loss/update spikes). We introduce **PAC-Opt**, a lightweight wrapper for standard stochastic optimizers that adds **per-band closed-loop control**: (1) an **Anti-Resonance Gate** that monitors band telemetry and rejects/attenuates band updates exhibiting narrowband spectral structure after robust detrending and high-pass filtering; and (2) a **Contraction-Stability Sentinel** that detects persistent local instability via robust secant smoothness estimates and triggers short band-wise safety episodes (temporary LR attenuation and optional fallbacks). A key feature is a calibration protocol that sets gate thresholds to achieve a user-specified false-reject rate on benign aperiodic dynamics (**single knob** α_{acc}), yielding parameter-free defaults that are not tuned on evaluation benchmarks. Synthetic resonance stress tests and realistic workloads with organic periodicities evaluate reductions in tail metrics (e.g., CVaR of loss spikes) without harming median performance. Deterministic aperiodic schedules (including prime-based instantiations used for reproducible calibration) perform no better than matched-random schedules *within* the closed-loop system, underscoring that the reactive architecture—not the schedule—is the primary contribution.

Positioning (what this is *about*)

- **Core contribution:** a *closed-loop control* wrapper for optimizers: **diagnose** → **decide** → **react**, per parameter band.
 - **Not a “primes are magic” claim:** prime-derived schedules are treated as one **deterministic aperiodic signal source** useful for *reproducible calibration* and as an optional schedule family.
-

Design commitments (v2.1 final)

1. **Band-local control:** rejects/attenuates only implicated parameter bands; avoids global stalls.
 2. **Nonstationarity-aware telemetry:** robust normalization + high-pass prior to spectral tests.
 3. **Gate is fast:** Welch PSD + spectral flatness as default; cepstrum only near threshold.
 4. **CSC is a sentinel:** event-triggered safety episodes, not a brittle per-step clamp.
 5. **Adoption-first defaults:** expose a single user knob α_{acc} ; publish universal defaults.
 6. **Reproducibility:** deterministic banding fallback uses **parameter name + shape** hashing.
-

Algorithm overview

We wrap a base optimizer (SGD-momentum, Adam, AdamW, ...). Parameters are partitioned into K **bands**. Each training step proposes some subset of bands B_t for updating (optionally via a schedule engine). For each band attempt, PAC-Opt:

1. computes **proposed update telemetry**,
2. updates the **anti-resonance gate** statistics,
3. accepts or rejects/attenuates the band update,
4. updates a **stability sentinel** score $\hat{\rho}$, and if instability persists, triggers a short safety episode.

The base optimizer remains the workhorse; PAC-Opt is a monitoring + control layer.

1) Bands and scheduling

1.1 Banding (defaults)

Default banding: each PyTorch `param_group` is a band.

Fallback banding: fixed-size chunks (e.g., 4096 scalars) assigned by a stable hash:

- hash key includes (**parameter name, parameter shape, chunk index**),
- band id = `hash(key) % K`,
- ensures identical banding across runs for the same model definition.

1.2 Scheduling (optional, secondary)

Scheduling can be “update all bands each step” or a band engine that proposes B_t . If a schedule engine is used, deterministic aperiodic schedules can be plugged in:

- $S \in \{\texttt{prime_mod}, \texttt{prime_gap}, \texttt{prime_density}\}$
- matched-random aperiodic controls with identical update counts.

Note: In the current framing, schedules are mainly used to (i) stress-test resonance and (ii) help calibrate gate acceptance constraints.

2) Telemetry pipeline (raw $\rightarrow$ detrend $\rightarrow$ log)

For band b at attempt t , define raw telemetry:

$$r_{b,t} := \|\Delta x_{b,t}\|$$

2.1 Robust detrending on the *raw* scale

Detrend before logging:

$$\tilde{r}_{b,t} := \frac{r_{b,t}}{\mathrm{median}(r_{b,t-W:t}) + \epsilon}$$

2.2 Stabilize scale

$$z_{b,t} := \log(1 + \tilde{r}_{b,t})$$

2.3 Micro-smoothing (optional)

A tiny smoothing reduces high-frequency noise prior to differencing:

$$\bar{z}_{b,t} := \mathrm{MA}_3(z)$$

2.4 High-pass filter

Cheap and effective:

$$y_{b,t} := \bar{z}_{b,t} - \bar{z}_{b,t-1}$$

Sampling note: buffers are indexed by **attempts** (only when the band is proposed), not by wall-clock step. This prevents penalizing bands during quiet periods.

3) Anti-Resonance Gate (Welch + Spectral Flatness + Cepstrum)

Maintain a rolling buffer of $y_{b,\cdot}$ with window length W (e.g., 128 attempts). Compute PSD $P[k]$ via **Welch's method**.

3.1 Stage 1 — Spectral Flatness (SF)

$$\mathrm{SF}_{b,t} := \frac{\exp\left(\frac{1}{K} \sum_k \log(P[k] + \epsilon)\right)}{\frac{1}{K} \sum_k (P[k] + \epsilon)} \in (0, 1]$$

Intuition: SF near 1 is noise-like/flat; small SF indicates line/comb structure.

3.2 Stage 2 — Cepstral combness (only near threshold)

If $\mathrm{SF} \in [\tau, \tau + \delta]$, compute cepstrum:

$$c[q] := \left| \mathrm{IFFT}(\log(P[k] + \epsilon)) \right|$$

Define combness on a quefrency band $\mathcal{Q}$ that excludes ultra-low quefrency (trend) and ultra-high (noise):

$$\mathrm{Comb}_{b,t} := \frac{\max_{q \in Q} c[q]}{\sum_{q \in Q} c[q]} + \epsilon$$

3.3 Gate decision

Accept band attempt iff:

- $\min_{W \in \mathcal{W}} \mathrm{SF}_{b,t}(W) \geq \tau$, and
- if near threshold then $\mathrm{Comb}_{b,t} \leq \theta$.

3.4 Action on reject

Band-local only:

- **hard reject:** LR multiplier $\rightarrow 0$ for that attempt, or
- **soft reject:** LR multiplier $\times \beta$ (often better continuity).

4) Contraction-Stability Sentinel (CSC as anomaly detector)

4.1 Robust secant smoothness estimate

Compute per-band secant estimates on attempts:

$$s_{b,t} := \frac{\|g_{b,t} - g_{b,t-1}\|}{\|x_{b,t} - x_{b,t-1}\|} + \epsilon$$

Aggregate robustly over the last m secants (Winsorized or Huberized mean):

$$\hat{L}_{b,t} := \mathrm{RobustAgg}(s_{b,t-m+1:t})$$

4.2 Stability indicator

$$\hat{\rho}_b(t) := 1 - \alpha^{\mathrm{base}}_{b,t} \hat{L}_{b,t}$$

Interpretation: a *smoothness/instability anomaly signal*, not a global convergence guarantee.

4.3 Event-triggered safety episodes (attempt-based)

Define an “attempt” as a step where band b is proposed ($b \in B_t$).

Trigger if $\hat{\rho}_b(t) > \rho_{\max}$ for k consecutive attempts. On trigger:

- attenuate LR multiplier by β for the next n attempts,
- optional fallback during the episode (e.g., gradient clipping).

Conservative default: $\rho_{\max} = 1.02$, $k = 3$, $n = 10$, $\beta = 0.5$, $m \approx 7$.

5.1 The user-facing knob

- α_{acc} : maximum allowed false-reject probability on *benign aperiodic* dynamics (e.g., 0.02).

5.2 Calibration suite (appendix-defined)

Must be clearly separated from evaluation benchmarks.

Requirements:

1. small, diverse tasks (vision + NLP + RL),
2. **no injected periodicities**, trained “normally,”
3. multiple base optimizers (SGD, Adam, AdamW).

5.3 Calibration output

The calibration procedure returns a universal tuple (τ, θ, δ) (and fixed PSD choices) satisfying:

$$\Pr(\text{reject} \mid \text{allowed}) \leq \alpha_{\text{acc}}$$

where “allowed” includes deterministic aperiodic schedules (e.g., prime-based), Sturmian-like aperiodics, and matched-random aperiodics.

Key defense: these defaults are *not* tuned on the test benchmarks.

6) Metrics and statistical protocol (tail-risk-first)

6.1 Tail metrics (operational definitions)

- Loss spike: $\Delta \ell_t = \max(\ell_t - \ell_{t-1}, 0)$
- Report $\text{CVaR}_{0.95}(\Delta \ell_t)$

Robust spike score for gradients:

$$s^{(g)}_t = \frac{\|g_t\|}{\|\text{median}(g)\|} + \epsilon$$

Report $Q_{0.95}(s^{(g)}_t)$ and $\text{CVaR}_{0.95}(s^{(g)}_t)$. Similarly for updates $\|\Delta x_t\|$.

6.2 Multiple comparisons control

Use **Benjamini–Hochberg** FDR across tasks $\times$ conditions $\times$ seeds $\times$ schedule variants.

7) Experiments (three-part story)

Experiment 1 — Mechanism (resonance stress test)

Synthetic quadratic + injected sinusoidal forcing with frequency sweep:

- show PAC-Opt flattens resonance peaks in tail metrics,
- ablations: gate-only vs sentinel-only vs full PAC-Opt,
- schedule variants inside PAC-Opt (prime vs matched-random): expect small/no difference.

Experiment 2 — Organic periodicity (realistic cyclicities)

Use settings with observed spectral lines:

- CIFAR with periodic reshuffle/augmentation cycle,
- NLP with periodic evaluation cadence,
- distributed simulation with periodic averaging/synchronization.

Demonstrate reduced tail risk (CVaR/quantiles) without degrading median metrics.

Experiment 3 — Scale + “do no harm”

Larger model (e.g., ViT-B / ImageNet subset) with **no injected periodicities**:

- overhead < ~5% target,
- null result on accuracy is positive (means gate doesn't harm benign runs).

8) Predictions (falsifiable)

- **Primary:** PAC-Opt (gate + sentinel) reduces tail-risk metrics under explicit periodic forcing and at least one organic periodicity setting, surviving BH-FDR.
- **Secondary:** within PAC-Opt, deterministic aperiodic schedules (prime-based included) perform no better than matched-random aperiodic schedules.

9) Minimal API sketch (illustrative)

```
class PACOptWrapper:
    def __init__(self, base_opt, bands, defaults):
        self.base = base_opt
        self.bands = bands # mapping: band_id -> list(params)
        self.gate = AntiResonanceGate(defaults)
    # tau/theta/delta derived from alpha_acc
    self.csc = CSCSentinel(defaults)
```

```

        self.lr_mult = {b: 1.0 for b in self.bands}

    @torch.no_grad()
    def step(self, closure=None):
        # 1) apply per-band multipliers to param groups (implementation-
        specific)
        apply_lr_multipliers(self.base, self.bands, self.lr_mult)

        # 2) base optimizer step
        loss = self.base.step(closure)

        # 3) per-band attempt updates
        for b in proposed_bands_this_step():
            r = estimate_update_norm(b)
            # ||Δx|| exact (debug) or reconstructed from state

            # telemetry pipeline: raw -> detrend -> log -> smooth -> difference
            self.gate.push_raw(b, r)

            # gate decision
            if not self.gate.accept(b):
                self.lr_mult[b] *= 0.0 # or *= beta for soft reject
                continue

            # sentinel update (attempt-based)
            rho_hat = self.csc.update(b)
            if self.csc.triggered(b, rho_hat):
                self.lr_mult[b] *= self.csc.beta

        return loss

```

10) Limitations and failure modes (explicit)

- **Nonstationarity mismatch:** regime shifts can still confound spectral tests; detrending/high-pass reduce but do not eliminate.
 - **Sparse attempts:** very infrequently updated bands yield weak PSD estimates; attempt-based buffering helps but may require longer windows.
 - **Telemetry measurement:** exact $\|\Delta x\|$ can be costly without state reconstruction; early prototypes may use debug snapshots.
 - **Adversarial alignment:** prime schedules are structured; the gate/sentinel is the real defense.
-

External references (foundational + related)

Spectral estimation / cepstrum

1. Welch, P. D. (1967). *The Use of Fast Fourier Transform for the Estimation of Power Spectra: A Method Based on Time Averaging Over Short, Modified Periodograms*. IEEE Trans. Audio and Electroacoustics.
2. Thomson, D. J. (1982). *Spectrum Estimation and Harmonic Analysis*. Proceedings of the IEEE. (Multitaper method.)
3. Bogert, B. P., Healy, M. J. R., & Tukey, J. W. (1963). *The Quefrency Analysis of Time Series for Echoes: Cepstrum, Pseudo-Autocovariance, Cross-Cepstrum, and Saphe Cracking*. In *Time Series Analysis* (Proc. Symp., Rosenblatt, ed.).

Spectral flatness / tonality 4. Johnston, J. D. (1988). *Transform Coding of Audio Signals Using Perceptual Noise Criteria*. IEEE Journal on Selected Areas in Communications. 5. Dubnov, S. (2004). *Generalization of Spectral Flatness Measure for Non-Gaussian Linear Processes*. IEEE Signal Processing Letters. (DOI: 10.1109/LSP.2004.831663) 6. ISO/IEC 15938 (MPEG-7): Multimedia Content Description Interface (Audio descriptors include spectral flatness / related measures).

Robust statistics 7. Huber, P. J. (1964). *Robust Estimation of a Location Parameter*. Annals of Mathematical Statistics.

Tail risk metrics 8. Rockafellar, R. T., & Uryasev, S. (2000). *Optimization of Conditional Value-at-Risk*. Journal of Risk. 9. Rockafellar, R. T., & Uryasev, S. (2002). *Conditional Value-at-Risk for General Loss Distributions*. Journal of Banking & Finance.

Multiple testing / FDR 10. Benjamini, Y., & Hochberg, Y. (1995). *Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing*. Journal of the Royal Statistical Society: Series B.

Base optimizers / common stabilizers 11. Kingma, D. P., & Ba, J. (2014). *Adam: A Method for Stochastic Optimization*. (arXiv:1412.6980) 12. Loshchilov, I., & Hutter, F. (2017). *Decoupled Weight Decay Regularization*. (AdamW; arXiv:1711.05101) 13. Pascanu, R., Mikolov, T., & Bengio, Y. (2013). *On the Difficulty of Training Recurrent Neural Networks*. ICML. (Gradient norm clipping.) 14. Loshchilov, I., & Hutter, F. (2016). *SGDR: Stochastic Gradient Descent with Warm Restarts*. (arXiv:1608.03983) 15. Smith, L. N. (2015/2017). *Cyclical Learning Rates for Training Neural Networks*. (arXiv:1506.01186; later appeared in WACV.)